GUIDE 43 · GOVERN

Give your team their own admin access

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=43-give-your-team-their-own-admin-access. This copy was made on 4 November 2026.

Time: 15 minutes. **You'll end up with:** everyone who manages QueryAPIGate signing in as themselves, with a role that fits what they do; CI on a token of its own; an audit log that says who changed what; and the shared `QUERYAPIGATE_API_KEY` retired to a break-glass key - or removed.

Until now, managing the server meant holding `QUERYAPIGATE_API_KEY`: everyone could do everything, and the audit log said `admin` for every change. Named administrators change that (ADR 0003). Data callers - scoped API keys, signed-in app users - aren't affected.

Every command and response below was run against a real server.

Step 1: Create the first owner

On the machine (or in the container) where the store is - no running server and no key needed:

```shell
queryapigate admins create alice --role owner --email alice@corp.com --label laptop
```

```text
Created administrator alice (owner).
Admin token tok_7c663440dc14 for alice (expires 2027-01-03) - store it now, it cannot be shown again:
  qagadm_9-2fLdbDffa6PaOdzKttS2U6j01oA9DiUoCawF3i8CE
Use it as the X-API-Key header, or paste it into the Console's key box.
```

In Docker: `docker compose exec queryapigate queryapigate admins create alice --role owner`. Tokens expire after 90 days unless you pass `--expires YYYY-MM-DD` (or `never`, for automation that rotates its own).

Alice signs in by pasting the token into the Console's sign-in page, the first thing a tab without one sees - the sidebar then says **Signed in as alice · Owner**, and **Sign out** beside it ends that - or by sending it as `X-API-Key`:

```shell
curl http://127.0.0.1:5000/api/v1/me -H 'X-API-Key: qagadm_9-2f...'
# {"name": "alice", "role": "owner", "via": "token", "capabilities": ["access.read", ...],
"data_access": true}
```

Step 2: Give each person a role

ROLE	FOR	MAY
`owner`	whoever runs the server	everything, including administrators
`admin`	the platform team	connections, API keys and roles, queries, cache, settings - not administrators
`developer`	people who write the APIs	saved queries and collections, SQL on every connection - no keys, no connection changes
`auditor`	security, compliance	read everything that describes the server (audit log, history, keys, connections) - no SQL at all

In the Console: **Access** → **Administrators** → **New administrator**. Their first token, valid for 90 days, is shown once - hand it over the way you'd hand over a password. Or with the API:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/administrators -H 'X-API-Key: qagadm_9-2f...' \
  -H 'Content-Type: application/json' -d '{"name": "bob", "role": "developer", "email": "bob@corp.com"}'
curl -X POST http://127.0.0.1:5000/api/v1/administrators/bob/tokens -H 'X-API-Key: qagadm_9-2f...' \
  -H 'Content-Type: application/json' -d '{"label": "laptop", "expires_at": "2027-01-03"}'
# {"id": "tok_2cfa12f4a781", "admin": "bob", "label": "laptop", "expires_at": "2027-01-03", ...,
#  "secret": "qagadm_WfBE..."}
```

What Bob's role allows is enforced by the server, whatever he uses to call it:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: qagadm_WfBE...' \
  -H 'Content-Type: application/json' -d '{"name": "partner", "connections": []}'
# 403 {"error": "The developer role may not create, change and delete API keys and roles",
#      "code": "role_forbidden", "capability": "access.write", "role": "developer", ...}
```

Bob can organise queries into collections, but not in a way that gives a key new reach: moving a query into the collection a partner's key holds is refused (`role_forbidden` , naming the key) - deciding who can call what is an owner's or admin's job. Publishing new versions of the queries partners already call is his.

In the Console, Bob simply doesn't see API keys, Roles, Settings or the buttons that change connections. He does see **Administrators**, showing his own account: every administrator issues and revokes **their own** tokens (a second laptop, a lost one), whatever their role. Someone else's tokens take an owner.

Step 3: Move automation to its own token

A pipeline is an administrator too - give it the narrowest role that works, and a token with an expiry you'll notice:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/administrators -H 'X-API-Key: qagadm_9-2f...' \
  -H 'Content-Type: application/json' -d '{"name": "ci-deploy", "role": "developer"}'
curl -X POST http://127.0.0.1:5000/api/v1/administrators/ci-deploy/tokens -H 'X-API-Key: qagadm_9-2f...'
\
  -H 'Content-Type: application/json' -d '{"label": "github-actions", "expires_at": "2027-04-01"}'
```

Store the `secret` in your CI's secret store and use it wherever the shared key was. A job that publishes saved queries (say, from a collection bundle) needs `developer`; one that also manages API keys needs `admin`.

Step 4: Read who did what

Every change now names the person and how they signed in:

```shell
curl 'http://127.0.0.1:5000/api/v1/audit' -H 'X-API-Key: qagadm_9-2f...'
```

```json
{"actor": "bob",   "via": "token", "action": "issue_admin_token", "target": "bob"}
{"actor": "alice", "via": "token", "action": "issue_admin_token", "target": "ci-deploy"}
{"actor": "alice", "via": "token", "action": "create_admin",      "target": "ci-deploy"}
```

`via` is `token` (their own), `break-glass` (the shared key), `cli` (a command on the server) or `open`.

Step 5: Retire the shared key

`QUERYAPIGATE_API_KEY` keeps working - as an owner - so nothing broke when you upgraded. Once an active owner exists, it is the **break-glass key**: each use is logged,

```text
WARNING queryapigate: The shared QUERYAPIGATE_API_KEY (break-glass key) was used for GET
/api/v1/connections
although named administrators exist
```

recorded in the audit log (`break_glass_used`), and raises an alert for 24 hours - **The shared admin key was used**

- on every instance's Alerts screen.

When nothing uses it any more (the alert stays quiet), take it out of the environment and restart. Authentication stays on - administrators exist - so the server doesn't reopen:

```shell
curl http://127.0.0.1:5000/api/v1/connections
# 401 {"error": "Unauthorized", "code": "unauthorized", ...}
```

Or keep it, sealed somewhere safe, for emergencies: the alert tells you if it's ever used.

If you're locked out

- **Someone lost their token** - they issue themselves another from a token they still have, or an owner does it for them (**Administrators → Tokens**), or anyone on the server runs:

  ```shell
  queryapigate admins token alice --label recovery --expires 2026-10-12
  ```

- **Someone leaves** - **Administrators → Edit → Active** off stops every token of theirs at once; or remove them (their name stays in the audit log).
- **The last owner can't be lost by accident**: without the shared key, demoting, deactivating or removing the only active owner is refused:

```text
409 {"error": "This would leave no active owner, and QUERYAPIGATE_API_KEY is not set - nobody could manage
administrators any more. Make someone else an owner first.", "code": "last_owner", ...}
```

- **Nobody can sign in at all** - `queryapigate admins list` and `queryapigate admins token NAME` on the server always work, because they use the store directly; so does setting `QUERYAPIGATE_API_KEY` again and restarting.

Good to know

- Tokens start with `qagadm_`, so a secret scanner can be given the pattern. Only their hash is stored.
- An administrator and an API key can't share a name (`409 name_taken`), and `admin` and `cli` are reserved - a name in the logs, `/metrics` and run history always means one caller.
- Owners, admins and developers can run SQL on every connection, as the shared key can. To give someone less than that, give them a scoped API key instead.
- Signing in to the Console through your company's identity provider (OIDC) is planned (BACKLOG #84 Phase 2); it will create administrators and map groups to these same roles.

Next steps

- Read the audit log to answer "who changed this, and when".
- Read the threat model - what a stolen token of each role can do.
- API reference: administrators and admin roles.